

Course Curriculum

Full-stack → AI Engineer roadmap · index of stages, modules and topics

7
stages

46
modules

215
topics

13
projects

~118h
study time

#	STAGE	MODULES	TOPICS	STUDY TIME
1	Python where it matters, and the backend gaps	8	38	19h 45m
2	LLM APIs and production prompting	6	33	17h 25m
3	RAG for real, and how to know it works	8	44	25h 15m
4	Tool use, agents, and MCP	7	34	19h 20m
5	AI system design, operations, and security	8	37	20h 45m
6	Capstone and portfolio	5	17	9h 40m
7	Interviews, depth, optionality	4	12	5h 50m

Stages are content units, not calendar months — no deadlines, no time boxes. Study time is the platform's per-topic reading estimate and excludes practice and project work. Modules are listed in the recommended order (in Stage 3, Evaluation comes before Retrieval quality: build the ruler first). **OPTIONAL** marks modules nothing else depends on.

Python where it matters, and the backend gaps

Build, test, containerise and deploy a typed, async Python API with Postgres and pgvector behind it.

s1.1	Python for AI code	2h 50m
s1.1.1	Decorators and what they are really doing	35 min
s1.1.2	Context managers and cleanup	30 min
s1.1.3	Generators and yield	35 min
s1.1.4	Type hints, generics and Protocol	40 min
s1.1.5	The traps that catch JavaScript developers	30 min
s1.2	Pydantic v2	2h 05m
s1.2.1	Models, validation, and getting data in and out	40 min
s1.2.2	Constraints, validators, and closed vocabularies	35 min
s1.2.3	Settings and secrets with pydantic-settings	25 min
s1.2.4	Why the whole AI ecosystem is built on this	25 min
s1.3	Async Python	3h 15m
s1.3.1	asyncio, and how it differs from the Node event loop	40 min
s1.3.2	gather, TaskGroup and cancellation	35 min
s1.3.3	Bounded concurrency with Semaphore	30 min
s1.3.4	httpx: timeouts, pooling, and retries with jitter	35 min
s1.3.5	Async generators and streaming	35 min
s1.3.6	Threads, processes and the GIL — in one page	20 min
s1.4	Tooling and testing	1h 40m
s1.4.1	uv, pyproject and never thinking about environments again	25 min
s1.4.2	ruff, and stopping arguments about formatting	15 min
s1.4.3	pytest, async tests, and faking HTTP	35 min
s1.4.4	Structured logging and not leaking secrets	25 min
s1.5	FastAPI	2h 55m
s1.5.1	Routes, models and documentation you get for free	30 min
s1.5.2	Dependency injection — the concept Express does not have	35 min
s1.5.3	Middleware, error handling and CORS	25 min
s1.5.4	Streaming responses with SSE	35 min
s1.5.5	Auth: API keys and JWT	25 min
s1.5.6	Testing a FastAPI app	25 min
s1.6	Postgres conversion	2h 30m
s1.6.1	MySQL to Postgres: what actually bites	30 min
s1.6.2	Reading EXPLAIN ANALYZE	30 min
s1.6.3	GIN, partial and expression indexes, and JSONB	25 min
s1.6.4	Full-text search with tsvector	30 min
s1.6.5	pgvector: vectors in your database	35 min

s1.7 Docker and local infrastructure 1h 55m

s1.7.1	Images, layers and a good Python Dockerfile	35 min
s1.7.2	docker compose: your whole stack in one command	30 min
s1.7.3	Environment, secrets and healthchecks	20 min
s1.7.4	Deploying a container somewhere real	30 min

s1.8 AI intuition — the cheap 8 hours 2h 35m

s1.8.1	What a token is, and why JSON costs more than prose	40 min
s1.8.2	Next-token prediction, and why models make things up	35 min
s1.8.3	What an embedding is, and what similarity means	40 min
s1.8.4	Temperature, attention, and how models get trained	40 min

Projects

p-1.1	The Port	Size S
p-1.2	Async Harvester	Size S

Readiness self-check (advisory, never a lock)

- A FastAPI endpoint that streams SSE while making several concurrent async calls with a cap — written without looking things up
- You can explain what pydantic is doing when validation fails, and why that error is useful
- docker compose up brings up your stack on a clean machine
- A pgvector table with an HNSW index exists, and you know what ef_search changes
- Tokens, embeddings and temperature — explainable without notes

STAGE 2

LLM APIs and production prompting

Build a reliable, streaming, cost-instrumented LLM feature against any of the three major providers, with structured output you can trust.

s2.1	What an LLM call actually is	3h 20m
s2.1.1	Your first call — to three providers	35 min
s2.1.2	Tokens, context windows and pricing in practice	35 min
s2.1.3	Sampling settings — and why the newest models took them away	30 min
s2.1.4	Stop reasons and usage: reading the response properly	30 min
s2.1.5	Thinking and effort: paying for reasoning on purpose	35 min
s2.1.6	Choosing a model, and living with rate limits	35 min
s2.2	Streaming, end to end	2h 40m
s2.2.1	How provider streaming works: events and deltas	30 min
s2.2.2	From provider to FastAPI to the browser	35 min
s2.2.3	Rendering streamed text in React without jank	35 min
s2.2.4	Cancel means cancel: stopping upstream billing	30 min
s2.2.5	Measuring and showing time to first token	30 min
s2.3	Production prompt engineering	4h 20m
s2.3.1	Anatomy of a system prompt	35 min
s2.3.2	Structure that helps: tags, documents and ordering	30 min
s2.3.3	Examples: the strongest signal in a prompt	35 min
s2.3.4	Reasoning: from 'think step by step' to thinking settings	30 min
s2.3.5	Decomposition: one prompt, one job	30 min
s2.3.6	Prompts are code: versioning and templates	30 min
s2.3.7	When prompts fail: the failure types and their fixes	35 min
s2.3.8	Testing prompt changes, and trading quality against cost	35 min
s2.4	Structured output you can trust	2h 35m
s2.4.1	Three ways to get structured output — and what each guarantees	35 min
s2.4.2	Pydantic as the contract	30 min
s2.4.3	Repair loops: fixing meaning, not syntax	35 min
s2.4.4	Closed vocabularies: enums, 'other' and 'unknown'	25 min
s2.4.5	Streaming structured output	30 min
s2.5	Reliability and cost	3h 05m
s2.5.1	Retryable versus fatal: reading provider errors	30 min
s2.5.2	Timeouts and the hung stream	25 min
s2.5.3	Fallback chains	30 min
s2.5.4	Prompt caching	35 min
s2.5.5	Cost accounting and budget guards	30 min
s2.5.6	Guardrails: input limits, personal data, and prompt injection basics	35 min

s2.6	Multimodal basics	OPTIONAL	1h 25m
s2.6.1	Images in prompts		30 min
s2.6.2	PDFs: send the file, or extract the text?		30 min
s2.6.3	Audio: transcription as a pipeline step		25 min

Projects			
p-2.1	Chat, done properly		Size M
p-2.2	Structured Extraction Service		Size M

Readiness self-check (advisory, never a lock)

- One interface, three providers, same feature
- Cancelling a stream actually stops upstream billing, provable in logs
- You can state what one request costs, from your own instrumentation
- Killing the primary provider key degrades gracefully instead of 500-ing
- A prompts/ directory with versions; any output traces back to a version
- You have hit five distinct prompt failure modes and know what fixed each

RAG for real, and how to know it works

Build retrieval over a messy real corpus, measure its quality with numbers, and improve those numbers deliberately.

Opener: Start by auditing the RAG project you already built

s3.1	The retrieval problem, properly	1h 40m
s3.1.1	Why retrieval exists	25 min
s3.1.2	RAG vs long context vs fine-tuning	35 min
s3.1.3	The eight stages — and auditing your own project	40 min
s3.2	Embeddings in practice	2h 50m
s3.2.1	Choosing an embedding model	35 min
s3.2.2	Distance metrics, normalisation and silent recall killers	35 min
s3.2.3	Batching, caching and the cost of re-embedding	35 min
s3.2.4	Seeing your corpus: projecting embeddings	30 min
s3.2.5	Shorter vectors and quantisation	35 min
s3.3	Ingestion — the unglamorous 60%	3h 40m
s3.3.1	PDF reality	40 min
s3.3.2	Every other format	35 min
s3.3.3	Cleaning: boilerplate, duplicates and broken text	35 min
s3.3.4	Metadata design	35 min
s3.3.5	Incremental sync	35 min
s3.3.6	The ingestion pipeline	40 min
s3.4	Chunking	4h 00m
s3.4.1	Fixed-size and recursive splitting	30 min
s3.4.2	Sizing chunks in tokens	25 min
s3.4.3	Structure-aware chunking	40 min
s3.4.4	Semantic and propositional chunking	30 min
s3.4.5	Parent-document and sentence-window retrieval	35 min
s3.4.6	Contextual retrieval	40 min
s3.4.7	Measuring chunk size on your corpus	40 min
s3.5	Vector storage	2h 50m
s3.5.1	pgvector in production	40 min
s3.5.2	HNSW and IVFFlat: the recall–latency dial	40 min
s3.5.3	Filtered search and the recall cliff	35 min
s3.5.4	Qdrant: the dedicated-engine contrast	35 min
s3.5.5	The rest of the landscape	20 min

s3.8	Evaluation	4h 10m
s3.8.1	Why evaluation comes first	25 min
s3.8.2	Building the golden set	40 min
s3.8.3	Retrieval metrics	35 min
s3.8.4	Generation metrics	35 min
s3.8.5	LLM-as-judge: rubrics, calibration and bias	40 min
s3.8.6	Evals in CI: regression gates	40 min
s3.8.7	Error analysis	35 min
s3.6	Retrieval quality: hybrid, reranking, query understanding	3h 25m
s3.6.1	Hybrid search with Reciprocal Rank Fusion	40 min
s3.6.2	Reranking	35 min
s3.6.3	Filters and permission-aware retrieval	35 min
s3.6.4	Query rewriting and multi-query	35 min
s3.6.5	HyDE, decomposition and step-back questions	30 min
s3.6.6	Diversity: MMR and de-duplication	30 min
s3.7	Context assembly	2h 40m
s3.7.1	The token budget	30 min
s3.7.2	Ordering and prompt layout	30 min
s3.7.3	Citations the UI can link to	40 min
s3.7.4	Memory and compaction in RAG chat	30 min
s3.7.5	When not to retrieve: routing	30 min

Projects

p-3.1	Document Intelligence, with receipts	FLAGSHIP	Size XL
p-3.2	Ask My Repo	OPTIONAL	Size M

Readiness self-check (advisory, never a lock)

- A hand-written golden set in the repo
- The eval suite runs in CI and you have watched it fail a deliberately bad change
- A before/after table: 6+ variants with recall@5, faithfulness, p95, ₹ per query
- You can argue RAG vs long context vs fine-tuning from your own numbers
- A hand error-analysis of 20 failures, grouped into categories
- You can separate how much hybrid search helped from how much reranking helped

Tool use, agents, and MCP

Build an agent that takes real actions safely, and publish an MCP server other people's AI tools can use.

s4.1	Tool calling, deeply	3h 20m
s4.1.1	The tool-calling round trip	35 min
s4.1.2	Designing tool schemas	40 min
s4.1.3	Parallel calls and tool choice	30 min
s4.1.4	Tool errors	30 min
s4.1.5	Validating arguments	35 min
s4.1.6	Tool result size	30 min
s4.2	The agent loop	2h 45m
s4.2.1	What makes an agent	25 min
s4.2.2	Writing the loop	45 min
s4.2.3	The four budgets	35 min
s4.2.4	Loop detection and logging	30 min
s4.2.5	How agents fail	30 min
s4.3	Memory and state	2h 15m
s4.3.1	Short-term memory: the working context	30 min
s4.3.2	Long-term memory	35 min
s4.3.3	Persistence and resumable runs	40 min
s4.3.4	User memory and privacy	30 min
s4.4	Orchestration patterns	2h 40m
s4.4.1	Chains and routers	30 min
s4.4.2	Fan-out: parallel calls	30 min
s4.4.3	Evaluator–optimiser and reflection	30 min
s4.4.4	Human in the loop	40 min
s4.4.5	Multi-agent systems — rarely	30 min
s4.5	One framework: LangGraph	2h 20m
s4.5.1	LangGraph: state, nodes and edges	40 min
s4.5.2	Checkpointing with Postgres	35 min
s4.5.3	Interrupts: pausing for a person	35 min
s4.5.4	Streaming, and the framework landscape	30 min
s4.6	MCP	2h 55m
s4.6.1	Why a protocol	25 min
s4.6.2	Tools, resources and prompts	35 min
s4.6.3	Using MCP servers from hosts	30 min
s4.6.4	Building an MCP server	50 min
s4.6.5	Transports, auth, security and A2A	35 min

s4.7

Agent safety and interface

3h 05m

s4.7.1

Sandboxing and least privilege

35 min

s4.7.2

Injection through tools and documents

40 min

s4.7.3

Audit logs

30 min

s4.7.4

The agent dashboard

40 min

s4.7.5

Evaluating agents

40 min

Projects

p-4.1

Support Ops Agent

FLAGSHIP

Size XL

p-4.2

Your MCP Server

Size M

- Readiness self-check

(advisory, never a lock)
- Your own agent loop, hand-written, with budget enforcement and loop detection

•

The same agent in LangGraph with Postgres checkpointing, resumable mid-run

•

An MCP server a stranger can install in one command

•

An approval gate that survives a server restart

•

An agent eval suite over 30+ scenarios, pass rate reported honestly

•

You can explain MCP vs A2A, and why multi-agent is usually the wrong default

AI system design, operations, and security

Design, deploy, observe and defend a real AI system — with the dashboards and incident write-ups to prove it.

s5.1	AI system design as a discipline	2h 25m
s5.1.1	A framework for AI system design	35 min
s5.1.2	Budgets first	35 min
s5.1.3	How AI system design differs	25 min
s5.1.4	A worked design, and eight to practise	50 min
s5.2	Serving architecture	2h 45m
s5.2.1	Workers, and the sync-inside-async trap	30 min
s5.2.2	Queues and workers	35 min
s5.2.3	Long-running work: jobs, polling, webhooks and progress	30 min
s5.2.4	Streaming at scale	35 min
s5.2.5	Rate limits: providers and tenants	35 min
s5.3	Caching, cost and routing	2h 45m
s5.3.1	The cache ladder	35 min
s5.3.2	Semantic caching, properly	35 min
s5.3.3	Model routing	35 min
s5.3.4	Unit economics and budget guards	30 min
s5.3.5	Latency work	30 min
s5.4	Observability	2h 40m
s5.4.1	What to trace	35 min
s5.4.2	Langfuse	35 min
s5.4.3	OpenTelemetry for AI calls	30 min
s5.4.4	The dashboards you need	30 min
s5.4.5	Feedback capture and alerting	30 min
s5.5	Evaluation as engineering	2h 35m
s5.5.1	Offline suites, versioned and sliced	30 min
s5.5.2	Assertions before judges	30 min
s5.5.3	Pairwise comparison	30 min
s5.5.4	Online evaluation: sampling, shadowing and canaries	35 min
s5.5.5	A/B testing AI features	30 min
s5.6	Security	2h 45m
s5.6.1	The OWASP Top 10 for LLM applications	40 min
s5.6.2	Prompt injection: the honest picture	30 min
s5.6.3	Data exfiltration	30 min
s5.6.4	PII, secrets and retention	30 min
s5.6.5	Multi-tenant isolation and abuse	35 min

s5.7 Deployment and CI/CD 2h 55m

s5.7.1	Containers and one cloud	35 min
s5.7.2	Managed data services	30 min
s5.7.3	A pipeline with an eval gate	40 min
s5.7.4	Staging, canary releases and rollback	30 min
s5.7.5	Load testing with k6	40 min

s5.8 Fine-tuning: when, not how **OPTIONAL** 1h 55m

s5.8.1	The fine-tuning decision	25 min
s5.8.2	SFT, DPO and LoRA, conceptually	30 min
s5.8.3	One small LoRA run — then stop	60 min

Projects

p-5.1	Production Hardening FLAGSHIP	Size XL
p-5.2	Break it and write it up	Size S
p-5.3	Eval harness library OPTIONAL	Size M

Readiness self-check (advisory, never a lock)

- One system on AWS or GCP, in containers, with CI/CD
- Tracing across all three projects, and a cost dashboard you can screenshot
- A CI eval gate blocking merges, demonstrated
- A load-test report with a real p95 and a documented breaking point
- Cost measurably reduced through caching and routing — you know the number
- Eight system designs whiteboarded out loud, two recorded and watched back
- OWASP LLM Top 10 reviewed against your own code, with real fixes committed
- One written postmortem

STAGE 6

Capstone and portfolio

Three flagship systems, and you are interviewing.

s6.1	Capstone — AI hiring copilot (you name it)	2h 25m
s6.1.1	Scoping the capstone	30 min
s6.1.2	Multi-tenancy, for real	40 min
s6.1.3	Explainable matching and a fairness review	45 min
s6.1.4	Real users: launch, listen, improve	30 min
s6.2	One specialisation	1h 35m
s6.2.1	Five specialisations, one choice	30 min
s6.2.2	Reading job descriptions like data	35 min
s6.2.3	Planning your specialisation project	30 min
s6.3	Portfolio	2h 05m
s6.3.1	The flagship README and its docs	35 min
s6.3.2	Demo videos that get watched	30 min
s6.3.3	GitHub, LinkedIn and a site that agree	35 min
s6.3.4	Writing (optional): claims with numbers	25 min
s6.4	Interview preparation	2h 40m
s6.4.1	The AI system design round	40 min
s6.4.2	The concepts round: answer from your projects	35 min
s6.4.3	Coding rounds: DSA plus AI-flavoured problems	45 min
s6.4.4	Take-homes, stories and the cost question	40 min
s6.5	Applications as a pipeline	55m
s6.5.1	Targets, referrals and outreach	30 min
s6.5.2	Tracking the pipeline	25 min

Projects

p-6.1 Capstone — AI hiring copilot (you name it) **FLAGSHIP** Size XL

Readiness self-check (advisory, never a lock)

- Capstone live, multi-tenant, real users, with a tenant-isolation test
- Three flagship projects with architecture docs, numbers and demo videos
- GitHub, LinkedIn and site consistent and evidence-led
- Twelve mock system designs, some with another person
- Applications out, pipeline tracked

STAGE 7

Interviews, depth, optionality

Offers — and a body of work that compounds either way.

s7.1 Interview execution and same-day debriefs 1h 15m

s7.1.1	Running the interview loop	25 min
s7.1.2	Same-day debriefs that close gaps	25 min
s7.1.3	When you don't know	25 min

s7.2 Depth on demand 1h 15m

s7.2.1	From a gap to a short plan	25 min
s7.2.2	Reading papers, specs and docs fast	25 min
s7.2.3	Small proof-of-skill builds	25 min

s7.3 Optional deep dives: voice, vLLM, GraphRAG, computer use OPTIONAL 2h 25m

s7.3.1	Voice agents: a latency problem first	35 min
s7.3.2	Self-hosting models with vLLM	40 min
s7.3.3	GraphRAG: when similar isn't enough	35 min
s7.3.4	Computer use and browser agents	35 min

s7.4 Negotiation and screening a role for real AI work 55m

s7.4.1	Screening a role for real AI work	25 min
s7.4.2	Negotiating an offer	30 min

Projects

p-7.1	Specialisation project	Size L
-------	------------------------------	--------

Readiness self-check (advisory, never a lock)

- Every round debriefed in writing the same day
- Each exposed gap closed before the next round